

RESEARCH ARTICLE

An Integrated UTM Solution for Modern Cybersecurity: Combining Deep Inspection, ML, and Policy Automation

SHREYASH HATKAR¹, KREET ROUT¹, AAYUSH LAD¹,
AND O. V. GNANA SWATHIKA², (Senior Member, IEEE)

¹School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India

²Centre for Smart Grid Technologies, Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India

Corresponding author: O. V. Gnana Swathika (gnanaswathika.ov@vit.ac.in)

This work was supported by Vellore Institute of Technology, India.

ABSTRACT The escalating frequency and sophistication of cyber threats necessitate a holistic and integrated approach to network security. This research proposes a comprehensive Unified Threat Management (UTM) system that consolidates key security functionalities—firewalling, intrusion detection and prevention, anti-malware protection, phishing detection, policy enforcement, and log analysis into a centralized, intelligent platform. By leveraging advanced technologies such as machine learning, deep packet inspection, and natural language processing (NLP), the proposed UTM system enhances real-time threat detection, reduces administrative complexity, and improves scalability for diverse network environments. A modular architecture facilitates adaptive protection, while cloud-based management enables dynamic policy updates and centralized monitoring. The system's effectiveness is demonstrated through the design and implementation of core modules, including an adaptive firewall, phishing detection engine using ensemble learning, and an NLP-powered security policy manager. Experimental evaluations validate the system's high detection accuracy and low false positive rates, showcasing its potential as a scalable and proactive defence mechanism against modern cyber threats. This study contributes a practical, intelligent UTM framework tailored for modern organizations seeking robust and simplified security infrastructure.

INDEX TERMS Unified threat management, cybersecurity, intrusion detection and prevention system, machine learning, phishing detection, deep packet inspection, firewall, anti-malware, natural language processing, security policy management, sandboxing, log analysis, network security, threat intelligence, signature-based detection.

I. INTRODUCTION

Cyber threats are changing at breakneck speed and target governments, organizations, and people. Malware such as ransomware, spyware, and trojans is still a primary attack vector, most frequently using software vulnerabilities and poor security settings [1]. Phishing and social engineering still trick people into giving away sensitive information, and Advanced Persistent Threats (APTs) allow for covert, long-term cyber espionage and sabotage [2]. The growth of IoT and cloud computing has brought new threats, with insecure devices and

misconfigured cloud services causing massive-scale DDoS attacks, data breaches, and unauthorized access [3]. Crypto-jacking and supply chain attacks have also grown, infecting systems for illegal cryptocurrency mining and injecting malicious code into trusted software [1]. Cybercrime financial losses are over \$600 billion per year, with ransomware attacks using double extortion techniques—encrypting data and threatening to release it [2]. Cybercriminals are also using AI and automation to increase attack accuracy, rendering conventional security measures useless. To mitigate these threats, organizations need to implement AI-based threat detection, zero-trust security frameworks, robust encryption, and real-time monitoring. Cyber awareness, active defence

The associate editor coordinating the review of this manuscript and approving it for publication was Ikramullah Lali.

mechanisms, and regulatory compliance are key to protecting digital ecosystems [3]. Unified Threat Management (UTM) consolidates various security functions—such as firewalls, intrusion detection and prevention systems, anti-malware, and content filtering—into a single, integrated platform, enhancing the efficiency and effectiveness of cybersecurity defences. The increasing frequency and complexity of cyber threats necessitate a uni-fied approach to threat management. For instance, the pro-liferation of advanced persistent threats (APTs) and zero-day vulnerabilities requires comprehensive security solutions capable of real-time threat detection and response. UTM systems address these challenges by providing centralized control and streamlined security operations, which are essential for protecting modern digital infrastructures [4]. More-over, the integration of multiple security functions within a UTM framework simplifies network management and reduces operational costs. This consolidation is particularly beneficial for small to medium-sized enterprises (SMEs) that may lack extensive IT resources but still face significant cybersecurity risks [5]. By adopting UTM solutions, organizations can achieve robust security postures without the complexity of managing disparate security tools. Legacy network security tools, such as firewalls, intrusion detection and prevention systems (IDPS), and antivirus tools, have long been the mainstay of protection against cyberattacks. But with the progress of cyber threats in terms of sophistication and the magnitude of attacks, the siloed nature of these tools has exposed a number of issues. For instance, there is no integration between different security products, and this results in fragmented visibility and response times that are slow to react to fast evolving threats [6]. Also, traditional solutions are never able to adjust to the fluidity of current cyber threats and thus become more vulnerable to zero-day attacks and advanced persistent threats [7]. This study is intended to create a Unified Threat Management (UTM) system that integrates several security features into one unified platform. The key goals are to improve threat detection and response, simplify operations, and enhance the scalability of security. By combining several security features, including firewalling, intrusion prevention, and malware detection, the envisioned UTM system aims to offer end-to-end protection from a broad spectrum of cyber threats. The envisioned contributions of this work are to design an effective UTM architecture, performance comparisons under different network scenarios, and a comparative study illustrating the merits of UTM over conventional multi-point security systems [4].

II. RELATED WORKS

Unified Threat Management (UTM) systems have become crucial for securing modern networks by integrating multiple security functions into a single platform. The primary advantage of UTMs is the consolidation of various security technologies—such as firewalls, intrusion prevention systems (IPS), antivirus, and content filtering—into one system, thus simplifying network security management. Research has

shown that UTMs are beneficial in managing and mitigating various cyber threats by providing comprehensive and layered security defences [8]. However, challenges remain in balancing the complexity of these systems with the need for high performance and low latency. As the threats facing networks become increasingly sophisticated, many researchers have focused on the performance and effectiveness of UTMs in tackling modern security challenges. In particular, research by Lau et al. evaluates the effectiveness of UTMs in combating advanced persistent threats (APTs) and zero-day attacks, which are difficult for traditional standalone systems to address [9]. Their study demonstrates that while UTMs can significantly reduce complexity, the integration of multiple security technologies into a single device may still leave certain vulnerabilities, particularly if the UTM lacks advanced adaptive threat detection mechanisms. Further examination of UTMs, particularly in the context of small and medium enterprises (SMEs), has been a focus of several studies. A significant challenge for SMEs is the implementation of complex security solutions that are both cost-effective and efficient. Zhang et al. conducted a comparative analysis of various UTM systems tailored for SMEs and found that the integration of machine learning into these systems could improve detection capabilities, reduce false positives, and help address new attack vectors [10]. They argue that future UTMs should combine traditional security measures with intelligent threat prediction mechanisms to provide better protection against emerging threats. Despite the advantages of UTMs, a major issue is the overhead associated with running multiple security functions simultaneously. Research by Bilgin and Bektas [11] found that performance degradation in UTM systems is a serious concern, particularly in high-traffic networks. Their study suggests that optimizing the resource allocation of UTM devices, while maintaining their effectiveness, is essential to achieving a balance between security and performance [11]. With the increasing sophistication of cyber threats, individuals rely on a combination of security solutions such as firewalls, intrusion detection and prevention systems (IDPS), and anti-malware software to protect their devices and sensitive information. Each of these solutions plays a distinct yet complementary role in enhancing cybersecurity resilience. Firewalls act as a basic security shield by screening incoming and outgoing traffic according to pre-defined rules, blocking unauthorized access and possible attacks [12]. Conventional firewalls are mostly concerned with packet filtering and access control; however, new developments, including the Fireman toolkit, optimize firewall performance by modelling and analysing rule sets to detect misconfigurations and security holes [12]. Although they are crucial, firewalls by themselves are not enough to combat advanced cyber-attacks, requiring complementary layers of protection. IDPS solutions augment firewalls by proactively scanning network traffic for suspicious behaviour and potential intrusions. These systems look at behavioural signatures and mark abnormalities, making them indispensable

for threat detection early on [13]. In contrast to firewalls that primarily implement static rule-based policy enforcement, IDPS solutions evolve dynamically to handle new attack patterns, enhancing security posture overall. Nonetheless, their usefulness tends to be highly dependent on the availability of data since live cyber threat intelligence is important in detection and prevention of evolving threats [14]. Anti-malware software continues to be an essential part of personal security measures, which are intended to identify and eliminate malicious programs like viruses, ransomware, and spyware. Conventional signature-based detection methods have been employed extensively, but they are ineffective against polymorphic malware and zero-day attacks [15]. To overcome these limitations, AI-based cybersecurity solutions have been incorporated into anti-malware tools, enhancing detection efficacy through behaviour-based and heuristic inspection [15]. Additionally, advanced tools like Pixy focus on identifying vulnerabilities in web applications, offering enhanced protection against web-based threats [16]. Biometric authentication integrated with cryptography-based security mechanism is another nascent trend among individual security products. Research has investigated how it is possible to enhance authentication techniques by combining biometric data, like fingerprints or iris scans, with encryption mechanisms while reducing password related risks [17]. Nevertheless, even with improved security, there are ongoing issues with storing biometric data and spoofing attacks. With evolving cyber threats, a layered security solution involving firewalls, IDPS, anti-malware with AI, and biometric authentication is necessary. Cybersecurity experts are stressing the need for ongoing research in these products to keep pace with emerging issues and provide adequate protection to citizens in a growing digital world [13]. Our solution addresses the limitations of traditional security systems by integrating various security features into a single Unified Threat Management (UTM) platform, providing comprehensive and centralized protection for networks. Unlike traditional standalone security tools, which often operate in silos and lack integration, our UTM solution consolidates key functionalities such as advanced malware protection, a robust firewall, live network monitoring, and phishing detection using cutting-edge machine learning models. Furthermore, our cloud-based architecture ensures seamless scalability, allowing for real-time updates and centralized management across multiple network endpoints. The inclusion of Natural Language Processing (NLP) for user-friendly policy implementation allows administrators to define security rules and policies through intuitive language commands, simplifying the configuration and reducing the complexity typically associated with traditional systems. This solution enhances threat detection and response capabilities, improving overall network security and operational efficiency while minimizing manual intervention. By providing a holistic, cloud-based, and intelligent approach, our UTM system overcomes the challenges posed by fragmented security solutions

TABLE 1. Technologies used and their purposes in the UTM model.

Technology	Usage
Java 17	Base programming language used to build the backend system.
Spring Boot	REST API creation and MongoDB integration; provides autoconfiguration, embedded server, and simplifies dependency management.
MongoDB	Persistent storage mechanism for various data types.
Maven	Dependency management and build tool for Java projects.
Netty	Captures data packets from the network for analysis.
IpTables	Implements firewall rules for traffic control.
OpenPhish	Provides phishing threat intelligence feeds for real-time detection.
SLF4J	Simple Logging Facade for Java to manage application logs.

and ensures proactive protection against evolving cyber threats.

III. PROPOSED UTM ARCHITECTURE

The Unified Threat Management (UTM) architecture integrates multiple security features into a single framework for effective network security. UTM combines multiple security mechanisms like firewall inspection, antivirus scanning, phishing intrusion prevention, and security policy enforcement to provide multilayered protection against dynamic cyber threats. UTM appliances aim to combine and streamline the security administration process while maintaining high performance and threat visibility for the network hardware. The complete data packet flow in the UTM system is depicted in Fig. 1, as a step by-step process of scanning and analysis from the input to the system to access in the internal network. It comprises packet filtering, IDS/IPS scanning, antivirus and sandbox analysis, and machine learning-based phishing protection mechanisms. Fig.2 shows modular structure of UTM architecture.

As summarized in Table 1, the different technologies used and their usage in the UTM model as a whole. Each technology plays a specific role in ensuring efficient system performance, security, and scalability. From backend development to network-level data handling and threat detection, these tools collectively contribute to the robustness of the UTM system. The integration of monitoring, logging, and containerization further enhances maintainability and deployment flexibility.

A. MODULE-1 ASDIPF FIREWALL (ADAPTIVE STATEFUL DEEP INSPECTION PACKET FILTER)

The increase in the sophistication and frequency of cyber threats has made it imperative for organizations to adopt stringent security measures that would safeguard their network and sensitive data. Firewalls stand at the very forefront of defence by overseeing, filtering, and controlling network traffic based on predefined security policies. Within

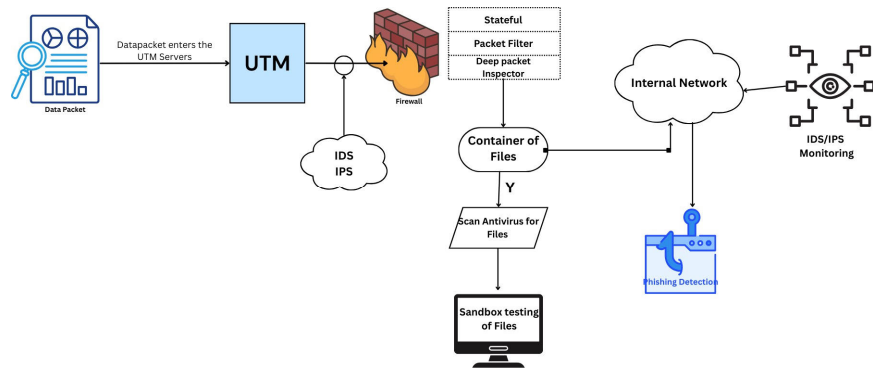


FIGURE 1. Detailed architecture of the proposed Unified Threat Management (UTM) system.

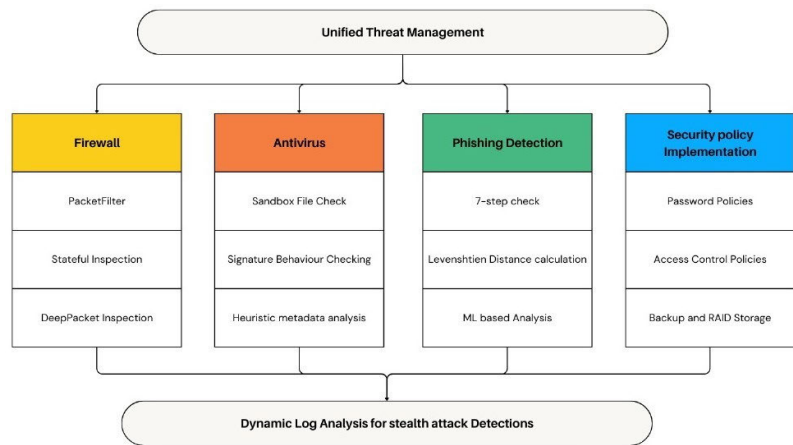


FIGURE 2. Common layout of UTM modules.

a UTM system, the firewall module is central to realizing comprehensive protection as it interfaces with other components of security, including intrusion detection/prevention systems (IDS/IPS), antivirus, and content filtering. This paper explores the role of firewalls in blocking unauthorized access, discusses the different types of firewalls, and explains their operational mechanisms. Additionally, we give one example of real-life implementation of the firewall in the UTM environment.

1) PACKET FILTER FIREWALL

Below is the Firewall’s step-by-step process flow, illustrated in Fig. 3. The Packet Filter dynamically enforces network access controls by executing appropriate commands using the device’s command-line interface. It is deployed with a combination of technologies that allow it to intercept and observe network traffic in real time. The filtering process is governed by custom firewall rules, which are based on attributes such as source IP address, destination IP address, port number, and the corresponding service. These rules are enforced programmatically using system-level commands. Additionally, the firewall rules are kept in a MongoDB database and controlled using a secure and modular RESTful API, which allows real-time updating and configuration of the rules. IP-based Filtering: Source IP filtering allows building firewall rules

defining a specific source IP address, therefore regulating traffic coming from that address. Based on the behaviour the rule defines, such traffic might be allowed or forbidden. Destination IP filtering is similarly defined as rules targeting particular destination IP addresses, therefore ensuring that the action of the rule affects only traffic headed towards these specified addresses. Port-based Filtering: Port-based filtering involves the use of rules that target specific ports. For instance, port matching allows rules to include a port parameter, ensuring that the rule applies only to traffic directed at a specified port, such as port 80. Alternatively, wildcard ports can be used, represented by special values like 0, allowing the rule to apply to traffic regardless of the target port. Action-based Filtering: Based on the designated action of the rule, action-based filtering controls traffic handling by the firewall. Should the action be set to” ALLOW,” the firewall lets traffic that fits the criteria. On the other hand, should the action be set to” DENY,” the firewall runs an iptables command to drop any traffic matching the rule.

2) DEEP PACKET INSPECTION FIREWALL

The Deep Packet Inspection (DPI) Firewall dynamically manages firewall rules via a RESTful API, captures network packets, and inspects their contents for known hostile signatures. It persistently stores firewall rules in a

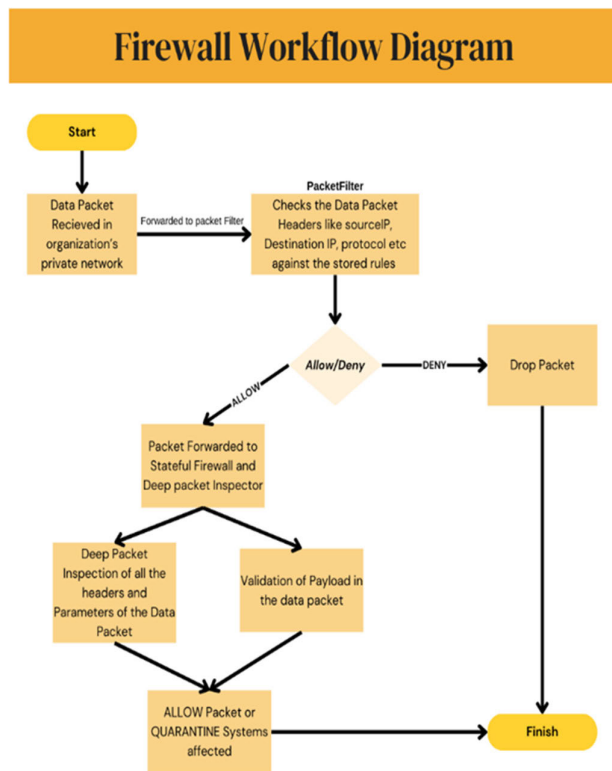


FIGURE 3. Detailed workflow of the firewall.

MongoDB database and blocks threats using a combination of signature-based detection techniques.

The Deep Packet Inspector Firewall is composed of the following components, as depicted in Fig.4:

- **Packet Sniffer:** Captures raw network packets from the network interface using libraries like Netty and forwards them for further analysis.
Key Technologies: Netty, Java NIO
- **Packet Analyser:** Processes raw packet data to extract payloads and critical header information (e.g., source IP).
Key Technologies: Java standard libraries (e.g., Arrays, InetAddress)
- **Signature Detector:** Analyses packet payloads against predefined malicious signatures using regex-based pattern matching.
Key Technologies: Java Regex, Pattern Matching
- **Alert Manager:** Triggers alerts (e.g., logging, notifications) and can invoke firewall actions such as blocking an IP upon detecting malicious signatures.
Key Technologies: Java Logging, System Command Execution (for iptables/nftables)

B. MODULE-2 PHISHING DETECTOR

Phishing is a common form of cyber threat that involves deceiving individuals by masquerading as credible entities to obtain sensitive information. Sophisticated phishing

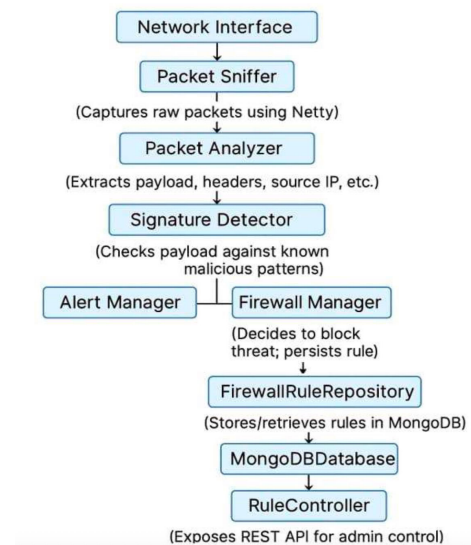


FIGURE 4. Components of the Deep Packet Inspection (DPI) Firewall.

attacks have rendered conventional security implementations ineffective, thus placing priority on automated detection systems. The methods examined in this study are machine learning-based phishing detection approaches tested against a dataset containing 11,430 URLs with 87 extracted features. The dataset is balanced with an equal number of phishing and legitimate URLs, ensuring unbiased evaluation of detection models.

1) NEED FOR PHISHING DETECTION

Phishing detection is critical in today's digital environment as it protects individuals and organizations from attacks that exploit human vulnerabilities. Phishing attempts are commonly initiated by sending fraudulent emails, creating fake web-sites, or using SMS scams. Due to attackers' increasingly sophisticated techniques, traditional security measures—such as spam filters and antivirus programs—are no longer sufficient. Advanced phishing detection techniques that include AI-based threat analysis, real-time URL scanning, and user awareness training are essential. A robust Phishing Detection System ensures a safer digital ecosystem, safeguarding transactions and digital assets from identity theft and data breaches.

2) DATASET

The dataset used in this work contains 87 features, categorized into three groups:

- URL Structure and Syntax: 56 features
- Web Page Content: 24 features
- External Service Queries: 7 features

This comprehensive feature set improves the machine learning model's ability to detect phishing URLs with greater accuracy and efficiency.

3) MACHINE LEARNING MODELS AND PERFORMANCE EVALUATION

Multiple machine learning algorithms were trained and validated using cross-validation techniques:

- **Logistic Regression:** Achieved a cross-validation accuracy of 0.9321.
- **Random Forest:** An ensemble learning method with an accuracy of 0.9614.
- **Gradient Boosting (XGBoost, LightGBM, CatBoost):** Averaging predictions from all three gradient boosting frameworks resulted in an accuracy of 0.969.
- **Support Vector Machine (SVM):** Using an RBF kernel, it achieved 0.9565 accuracy.
- **K-Nearest Neighbours (KNN):** Scored 0.9510 but exhibited slower performance on high-dimensional data.

A further improvement in accuracy was achieved using ensemble learning by soft voting Random Forest, Gradient Boosting (XGBoost, LightGBM, Catboost), which resulted in an outstanding accuracy of 97%. This demonstrates the effectiveness of combining diverse models to leverage their individual strengths.

4) SEVEN-STEP CHECK TO VALIDATE THE DOMAIN

Fig. 5 illustrates a seven-step process designed to validate and assess the phishing risk of domains or URLs using a multi-layered strategy that integrates heuristic checks and external threat intelligence.

- 1) **Google Safe Browsing API:** Checks the URL against a database of known malicious sites. If found, the phishing risk score is increased.
- 2) **VirusTotal Integration:** Submits the URL to VirusTotal for a consolidated malware detection report.
- 3) **OpenPhish Comparison:** Matches the input URL against a list of confirmed phishing domains.
- 4) **URL Pattern Analysis:** Identifies suspicious patterns like excessive special characters or IP-based URLs.
- 5) **Keyword Heuristic Analysis:** Detects suspicious keywords such as *login*, *verify*, *secure*, *account*, and brand names like *paypal* or *amazon*.
- 6) **Levenshtein Distance Matching:** Checks similarity to legitimate domains using the Levenshtein distance algorithm to detect typosquatting.
- 7) **Risk Scoring and Action:** Aggregates all previous steps to assign a phishing probability score. Based on thresholds, it classifies the URL as safe, suspect, or malicious.

If the risk score exceeds a critical threshold, the PolicyUp-dater Module enforces appropriate actions such as:

- Blocking access via firewall rules (e.g., iptables)
- Sending alerts to system administrators
- Updating local threat databases

All risk assessments and actions are logged for auditing, and alerts are generated as needed. This structured and intelligent approach, combined with machine learning, ensures high reliability in identifying phishing threats—even newly created

or previously unseen ones. The integration of real-time intelligence with heuristic evaluation provides a proactive and adaptive phishing detection framework.

C. MODULE-3 SECURITY POLICY

Security policies describe the rules and guidelines used by an organization to ensure the protection of critical business data and resources. They are pivotal to access control, customization, and enforcement mechanisms ensuring access to critical systems only by users who are authorized to do so while adhering to regulatory standards. With the maturity of Natural Language Processing (NLP), organizations can now more easily draft, enforce, and interpret security policies, allowing a more user-friendly and adaptive approach. This paper therefore considers the various roles that security policies play in regard to access control, customization, and enforcement, with cited examples of policy implementation and the application of NLP for policy management.

1) SECURITY POLICIES ON ROLE OF ACCESS CONTROL

Access control is the very essence of cybersecurity. It addresses two major questions: who accesses what resources, and under what circumstances? Security policies define access control parameters through:

- 1) **Authentication Mechanisms:** Policies define alternatives used for identifying users, for instance, passwords, multifactor authentication (MFA), or biometric techniques.
- 2) **Authorization Rules:** Policies state the extent to which a user has permissions based on their roles, responsibilities, and clearance levels; for example, an administrator would have greater access than a normal employee.
- 3) **Least Privilege Principle:** Policies that delegate the least privileges to guarantee the minimum access required by users to perform their tasks.
- 4) **Audit Trails:** Policies secure logging and monitoring of attempted access to detect unauthorized actions and ensure accountability.

Clarity in these contexts allows for a structured registry on access management aimed at preventing insider threats and external intrusions.

2) SECURITY POLICIES CUSTOMIZATION

One-size-fits-all policies are unsuitable for all security measures. Since organizations may operate in diverse environments and with varied requirements, it becomes vital to adapt customized policies to their specific needs. Common customization aspects include:

- 1) **Role-Based Access Control (RBAC):** Policies enacted according to operational hierarchies, assigning specific access rights based on roles. For instance, in healthcare, access to patient data is restricted to medical personnel, while billing staff may access only financial records.
- 2) **Context-Aware Policies:** Tailoring policies to contextual factors such as location, time, and device type.

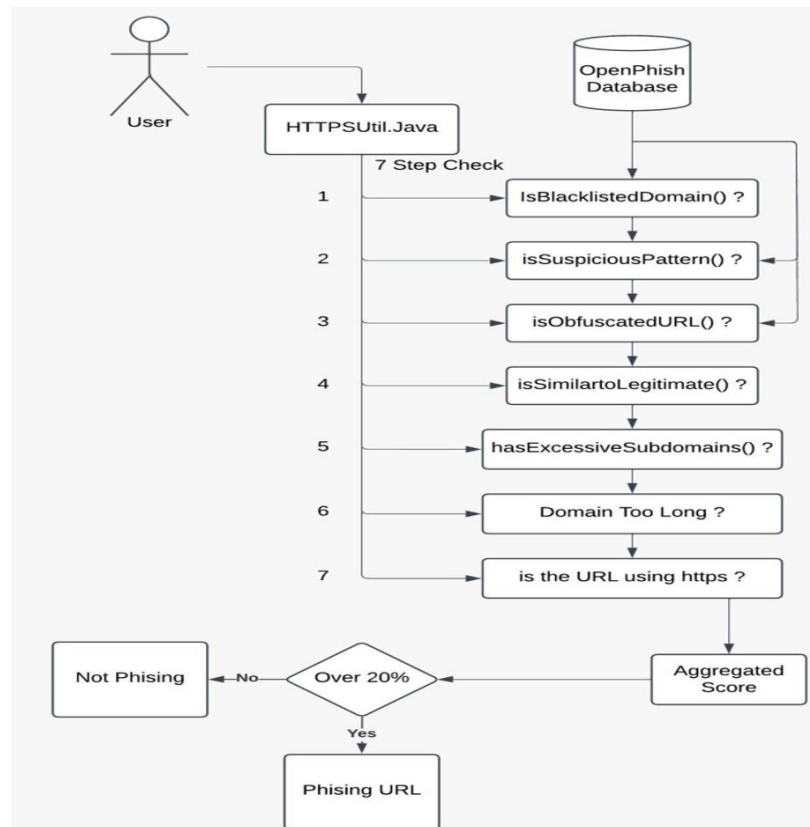


FIGURE 5. 7-step check for phishing detection.

Sensitive system access, for example, may be blocked after hours or from untrusted networks.

- 3) Regulatory Compliance: Ensuring policies align with industry regulations—such as GDPR for data protection or HIPAA for healthcare—to maintain legal compliance.
- 4) User Preferences: Policies may allow limited user-driven customization, such as selecting preferred MFA methods. Such customization ensures that security policies remain relevant, practical, and effective across varied organizational challenges.

3) ENFORCEMENT MECHANISMS OF SECURITY POLICIES

The effectiveness of a security policy hinges on the robustness of its enforcement mechanisms, which guarantee compliance and deter violations. Modern enforcement employs advanced methods including:

- 1) Real-time Monitoring: Monitoring tools track user activity and flag anomalies, enabling prompt remedial actions. For example, alerts or temporary account locks may follow suspicious logins from foreign countries.
- 2) Automated Enforcement: Tools like Identity and Access Management (IAM) automate user verification and manage permissions to prevent unauthorized access.

- 3) Penalties for Policy Violation: Clear consequences—such as account lockout or privilege revocation—reinforce adherence to policies.
- 4) Machine Learning Integration: ML algorithms are employed to analyse behavioural patterns and enforce adaptive security measures.

4) USE OF NLP

Creating security policies has traditionally been a complex and lengthy process, requiring proficiency in cybersecurity, regulatory compliance, and organizational workflow. Manual policy generation often led to inconsistencies, ambiguities, and inefficiencies. However, the advent of Natural Language Processing (NLP) has revolutionized this process, introducing automation, intelligence, and scalability. NLP systems can analyse vast quantities of unstructured text—such as legal documents, industry standards, and internal policies—to extract relevant information for coherent and actionable policy creation. This section explores how NLP facilitates policy generation, including its components and real-world applications.

D. MODULE-4 IDPS

As cyber threats become increasingly complex, organizations require more sophisticated tools to detect and respond to malicious activities swiftly. An Intrusion Detection and

Prevention System (IDPS) is a proactive mechanism designed to identify, inspect, and terminate active intrusions before any payload can execute. Within a Unified Threat Management (UTM) framework, the IDPS plays a pivotal role by integrating security tools such as firewalls, antivirus programs, and content filters. This integration provides a holistic and interoperable solution for comprehensive protection. This paper highlights the necessity of IDPS, the underlying technologies such as signature and anomaly detection, and how it integrates with UTM systems.

1) NEED FOR IDPS

Given the prevalence and complexity of modern cyberattacks, organizations must adopt IDPS solutions for adequate threat response. The primary motivations for implementing IDPS include:

- 1) **Real-time Detection of Threats:** IDPS offers the capability of monitoring network traffic and system activities around the clock to detect suspicious behavior or known attack patterns.
- 2) **Proactive Defense:** Unlike standard and traditional firewalls that filter traffic based on defined rules, IDPS actively monitors to detect and remediate threats, thus providing added protection.
- 3) **Regulatory Compliance:** Regulations such as GDPR, HIPAA, and PCI DSS mandate or recommend the use of IDS systems to protect classified data.
- 4) **Reduced Downtime:** Early detection and blocking of attacks by IDPS help avert service disruptions and reduce potential losses.
- 5) **Visibility into Network Environment:** IDPS provides detailed logging and reporting capabilities, allowing administrators and CSIRT teams to gain a clear overview of the network and improve incident response strategies.

These capabilities make IDPS a basic and crucial apparatus that firms require if they intend to secure their networks against both known as well as emerging threats.

E. MODULE-5 ANTIVIRUS AND SANDBOXING

Viruses or malware remain among the most devastating cyber threats, capable of compromising sensitive data, disrupting business operations, and causing significant financial losses. To defend against such threats, organizations employ various advanced malware protection tools, such as sandboxing, which isolates suspicious files and applications within a virtual environment for analysis. Effective malware detection and prevention require a multi-layered approach that combines traditional techniques with modern technologies. These measures aim to defend against both known and unknown threats, thereby ensuring comprehensive security against a wide spectrum of attacks.

1) SIGNATURE-BASED DETECTION

Signature-based detection is one of the oldest and most widely adopted techniques for identifying malware.

It involves comparing files and applications against a database of known malware signatures. When a match is found, the file is flagged as malicious. This method is highly effective in detecting well-known threats and produces quick and accurate results.

However, this approach is limited when it comes to detecting zero-day or unknown threats, as it depends solely on existing signature databases. Regular updates are necessary to maintain its effectiveness. Moreover, cybercriminals often employ evasion techniques such as polymorphism and obfuscation to bypass signature-based systems.

2) BEHAVIOURAL ANALYSIS

Behavioural analysis provides a more adaptive, heuristic approach to malware detection. Instead of depending on predefined signatures, this method observes the real-time behaviour of files and applications to identify potentially malicious actions. For instance, if a file attempts to modify system settings, access sensitive data, or initiate unauthorized outbound connections, it may be flagged for suspicious behaviour. Behavioural analysis is particularly effective against advanced threats like fileless malware, ransomware, and polymorphic viruses—many of which can evade signature-based systems. However, this technique may generate false positives if legitimate software exhibits unusual behaviour. Additionally, it demands considerable computational resources to monitor system activities continuously.

3) SANDBOXING

Sandboxing has become a cornerstone of modern malware protection. It allows organizations to analyse potentially harmful files and applications without risking the production environment. A sandbox is an isolated virtual environment where suspicious code can be executed, monitored, and analysed safely. During sandbox execution, any malicious behaviour remains confined within the sandbox and can not impact the host system or broader network. Security teams can closely observe actions such as file modifications, registry changes, or network communication attempts. By providing detailed insights into threat behaviour, sandboxing ensures that even sophisticated malware is neutralized before it can affect critical infrastructure. This isolated approach not only strengthens detection capabilities but also enhances the overall resilience of cybersecurity systems.

F. MODULE-6 LOG ANALYSIS AND REPORTING

Log analysis is pivotal in cybersecurity. Logs furnish visibility into an organization's IT infrastructure by detailing activities across networks, servers, applications, and endpoints. This visibility is critical for detecting suspicious behaviour that may indicate a cyberattack. For example, repeated failed login attempts or unusual outbound traffic

could serve as early warnings of a brute-force attack or a data exfiltration attempt.

Moreover, logs play a crucial role in incident response. When a breach occurs, logs provide an accurate historical account of the events leading up to the incident. This information helps security teams understand the attack vector, assess the impact, execute remediation measures, and support recovery efforts. Regulatory frameworks such as GDPR, HIPAA, and PCI-DSS often mandate log collection and analysis to ensure accountability and transparency. Frequent audits of these logs help organizations demonstrate compliance and avoid legal penalties, operational disruption, and reputational damage.

1) SECURITY AND PERFORMANCE MONITORING

Log analysis aids not only in security but also in performance monitoring and optimization. With log data, users can identify system bottlenecks, outages, or resource overuse that impact system efficiency. For instance, logs from web servers may reveal page load delays or high error rates, allowing IT teams to resolve issues proactively. Similarly, logs from network devices may highlight congestion or misconfiguration problems causing connectivity issues. Logs also offer insights into user behaviour. By analysing access logs, organizations can detect unauthorized access attempts or anomalies, such as employees accessing sensitive data outside regular hours. These insights enable enforcement of access control policies and mitigation of insider threats.

2) LOG AGGREGATION AND ANOMALY DETECTION

The process of log analysis typically begins with log aggregation—collecting and centralizing log data from diverse sources using platforms like Security Information and Event Management (SIEM) systems. Once aggregated, logs are normalized into a standard format for efficient processing and analysis. Pattern recognition techniques are then applied to detect deviations from established behavioural norms. Machine learning algorithms can analyse historical log data to establish baselines and raise alerts upon detecting anomalies, such as unusual login times, logins from unexpected geographical locations, or sudden spikes in data transfers. Correlation analysis further enhances detection capabilities by linking events across systems that may initially appear unrelated, enabling a comprehensive understanding of potential threats.

3) REPORTING AND VISUALIZATION

Reporting is essential for translating raw log data into actionable intelligence. Reports consolidate findings from log analysis, highlight trends, anomalies, and potential risks. Daily security reports may include information on blocked intrusion attempts, flagged anomalies, and compliance issues. These reports are often visualized using dashboards and graphs to enhance accessibility for both technical and non-technical stakeholders.

IV. IMPLEMENTATION AND METHODOLOGY

A. FIREWALL PACKET FILTER DOCUMENTATION

The Firewall Packet Filter is a Java-based application designed to dynamically enforce network access rules by executing Linux iptables commands. It offers a REST API that allows users to create, update, and delete firewall rules, which are persistently stored in MongoDB. The system uses Spring Boot for backend development, Netty for handling network events, and SLF4J Logback for logging and diagnostics. By leveraging iptables, the application applies firewall rules directly at the OS level, making it efficient and flexible for managing network traffic.

1) FILTERING SCHEMES

The packet filter enforces rules using the following filtering schemes:

- IP-Based Filtering
 - *Source IP Filtering*: Blocks or allows traffic from specific source IP addresses.
 - *Destination IP Filtering*: Controls traffic directed to particular destination IP addresses.
- Port-Based Filtering
 - *Port Matching*: Allows or blocks traffic to specific ports (e.g., 80 for HTTP).
 - *Wildcard Port*: Applies rules to all ports using a wildcard (e.g., port 0).
- Action-Based Filtering
 - *ALLOW*: Executes an iptables command with `-j ACCEPT`.
 - *DENY*: Executes an iptables command with `-j DROP`.
- Command Execution
 - Utilizes a `CommandExecutor` class to run iptables commands via the operating system terminal.

2) FLOW AND OPERATION

The firewall system follows a structured flow for rule management and enforcement:

- 1) **User Interaction**: Clients use tools like Postman, curl, or a frontend UI to send HTTP requests to the REST API.
Example Request:

```
POST /api/firewall/rules Body: {
  "sourceIp": "10.175.149.178,"
  "destinationIp": "192.168.157.134,"
  "port": "80,"
  "action": "DENY"
}
```
- 2) **Controller Layer**:
 - `FirewallController` receives and parses the JSON payload.
 - Forwards the rule data to `FirewallService`.
- 3) **Service Layer**:

- Handles persistence and business logic.
- Constructs the command:
 - For DENY:


```
sudo iptables -A INPUT -s 10.175.149.178
-d 192.168.157.134 -p tcp
-dport 80 -j DROP
```
 - For ALLOW:


```
sudo iptables -A INPUT -s 10.175.149.178
-d 192.168.157.134 -p tcp
-dport 80 -j ACCEPT
```
- Executes the command via the `CommandExecutor`.

4) Command Execution:

- `CommandExecutor` runs the command using `Runtime.getRuntime().exec()`.

5) MongoDB Repository:

- `FirewallRuleRepository` stores rules in the `firewall_rules` collection.

6) System-Level Enforcement:

- Linux `iptables` firewall enforces traffic control as per defined rules.

7) API Response:

- On success: Returns a confirmation message (e.g., *"Firewall rule added successfully"*).
- On failure: Returns an error message.

B. DEEP PACKET INSPECTION (DPI) FIREWALL

The Deep Packet Inspection (DPI) Firewall is a Java-based application designed to capture and inspect network packets, detect malicious content using signature matching, and dynamically manage firewall rules. It offers a RESTful API to add, remove, and retrieve firewall rules, which are persistently stored in MongoDB. The firewall can optionally extend its functionality with Netty for packet capture and `iptables` for dynamic firewall rule enforcement. The architecture is modular, allowing easy integration of additional security features.

1) TECHNOLOGIES USED

The DPI Firewall leverages a combination of modern frameworks and tools for efficient packet processing and rule management. The core logic is implemented in Java, providing stability and performance. The Spring Boot framework is used to develop the RESTful API and manage dependency injection, while Spring Data MongoDB enables seamless integration with MongoDB for storing firewall rules. The project uses Maven for build and dependency management, allowing for easy inclusion of required libraries. MongoDB ensures scalable and reliable persistence of firewall rules. For deep packet inspection, the project can optionally use Netty, a non-blocking I/O framework, to capture and process raw network packets. Real-time firewall rule enforcement, if enabled, is managed via dynamically constructed and executed `iptables` commands. SLF4J and Logback are used for

runtime diagnostics and logging, allowing for robust monitoring.

2) SYSTEM ARCHITECTURE AND COMPONENTS

The DPI Firewall follows a modular architecture with components for packet inspection, rule management, and enforcement. The `Main.java` class is the entry point, initializing the Spring Boot server and starting the firewall service. REST endpoints are exposed via `RuleController.java`, which handles HTTP requests to add, delete, or fetch rules. The core business logic resides in `FirewallManager.java`, which manages CRUD operations and executes `iptables` commands when dynamic enforcement is enabled. Firewall rules are stored as MongoDB documents via `FirewallRule.java`, and data access is managed by `FirewallRuleRepository.java` using Spring Data MongoDB. Application-level configurations such as the MongoDB URI and server port are defined in `application.properties`, enabling the system to operate in a portable and configurable manner.

3) FLOW AND OPERATION

The DPI Firewall performs the following steps for rule management and enforcement:

- 1) **Client Interaction:** Clients (e.g., Postman or `cURL`) send HTTP requests with JSON payloads specifying the rule type (e.g., BLOCK or ALLOW), protocol, and port.
- 2) **Controller Handling:** `RuleController` parses the request and forwards it to `FirewallManager`.
- 3) **Rule Persistence:** `FirewallManager` stores the rule in MongoDB using `FirewallRuleRepository`.
- 4) **Command Execution (Optional):** If dynamic enforcement is enabled, the corresponding `iptables` command is constructed and executed. Examples:
To block TCP traffic on port 443:

```
sudo iptables -A INPUT -p tcp -dport 443 -j DROP
```


To allow TCP traffic on port 443:

```
sudo iptables -A INPUT -p tcp -dport 443 -j ACCEPT
```
- 5) **API Response:** The API responds with success or error messages depending on the outcome.

4) OPTIONAL DEEP PACKET INSPECTION (DPI) FUNCTIONALITY

If DPI features are enabled, the firewall inspects network traffic in real-time. This is achieved through the following components:

- **PacketSniffer:** Captures raw packets using Netty or Java NIO.
- **PacketAnalyzer:** Extracts metadata such as IP addresses, ports, and payload.
- **SignatureDetector:** Uses regex-based matching to identify malicious content.

- **AlertManager:** Triggers alerts and optionally enforces new firewall rules if threats are detected.

This extension allows for traffic filtering not only based on IP and port but also on payload content, significantly enhancing the firewall's ability to detect and mitigate security threats.

5) MONGODB INTEGRATION

MongoDB acts as the persistent layer for storing firewall rules. Each document in the firewall rules collection contains a unique ID and a rule string (e.g., "BLOCK TCP 443"). Spring Data MongoDB handles object-document mapping and provides builtin methods for CRUD operations, simplifying development and ensuring reliable data management.

C. WORKING FLOW OF UTM MODULES

The proposed Unified Threat Management (UTM) system operates through a modular yet integrated workflow that systematically analyses and filters incoming data packets to ensure comprehensive network protection as illustrated in Fig. 1 and Fig. 2. Initially, a data packet enters the UTM server, where it is routed through a series of core security modules. The first line of defence is the firewall, which implements stateful inspection, packet filtering, and deep packet inspection (DPI) to examine the structure and content of the packet. If the packet contains files or payloads, they are forwarded to a dedicated container module for further scrutiny.

Within the container, files are subjected to antivirus scanning to detect known malware signatures. If the files pass the antivirus stage, they proceed to sandbox testing, where they are executed in an isolated environment to observe any suspicious behaviour indicative of zero-day or polymorphic threats. Concurrently, all traffic is monitored by an integrated Intrusion Detection and Prevention System (IDS/IPS) that tracks anomalies and enforces security policies in real time. Phishing detection is handled separately via a dual-layered engine that combines traditional rule-based filtering with machine learning classifiers to assess the legitimacy of embedded URLs and email content. Clean and verified packets are then allowed into the internal network, which continues to be monitored for lateral movement or postintrusion activity using IDS/IPS mechanisms. This layered workflow ensures a balance between proactive threat mitigation and minimal disruption to legitimate network operations.

V. CHALLENGES AND LIMITATIONS

Despite the comprehensive design and robust functionality of the proposed Unified Threat Management (UTM) system, several challenges and limitations were encountered during development and evaluation. One of the primary technical challenges lay in integrating multiple security modules—such as firewalls, phishing detection, IDPS, and malware analysis—into a unified and scalable architecture without introducing system-level conflicts or excessive complexity. Ensuring seamless communication between modules

while maintaining modularity and real-time responsiveness required careful orchestration of services and data flow.

Additionally, performance limitations were observed during stress testing. For instance, enabling deep packet inspection and phishing ensemble inference simultaneously increased packet processing latency from a baseline of 1.5 ms to 5.1 ms under a 1000 Mbps load. CPU utilization also spiked to 75%, and memory usage to 70%, indicating the need for further optimization in high-throughput environments. The phishing detection system, while highly accurate on benchmark datasets (97%), showed a false negative rate of up to 8.2% on adversarial test samples crafted using obfuscation and typosquatting, suggesting reduced generalizability against novel phishing techniques. The system's dependence on third-party APIs like VirusTotal and OpenPhish also introduces vulnerabilities. Simulated outages of OpenPhish resulted in a 22% reduction in phishing detection coverage, and API rate limits may constrain large-scale deployments unless premium tiers are adopted. Furthermore, the NLP powered policy engine, though user-friendly, exhibited a misinterpretation rate of 12% in test cases involving complex or ambiguous sentence structures. These misconfigurations could potentially expose the system to privilege drift or unauthorized access.

Performance trade-offs also emerged, particularly due to the simultaneous operation of multiple resource-intensive components. Deep packet inspection, behavioural malware analysis, and real-time machine learning inference significantly increased the system's processing overhead, leading to potential latency in high-traffic environments. Balancing detection accuracy with system throughput was a critical concern, necessitating optimization of rule sets, caching mechanisms, and asynchronous processing where possible.

In terms of detection techniques, while the phishing module demonstrated high accuracy through ensemble learning, its performance still relies heavily on the quality and diversity of the training dataset. Newly crafted phishing domains and zero-day threats can sometimes bypass detection due to limited model generalization. Similarly, signature-based detection used in malware analysis and DPI remains vulnerable to obfuscation techniques and polymorphic malware, which can evade static analysis methods.

Additionally, possible security loopholes exist due to the reliance on external services (e.g., VirusTotal, OpenPhish, Google Safe Browsing API) for threat intelligence. Service outages, API rate limits, or false trust in third-party data sources could hinder timely and accurate threat classification. Furthermore, while NLP enhances usability in policy definition, improper parsing or ambiguous inputs may result in misconfigured rules, potentially exposing the system to unauthorized access or policy drift. These limitations underscore the need for continuous evaluation, threat model updates, and layered fallback mechanisms to maintain robust security in dynamic environments. The performance and efficiency of the Unified Threat Management (UTM) system are

measured by using several key performance indicators. They are required to assess the capability of the system to block, detect, and react to all kinds of cyber attacks as well as utilize the resources in the optimal way.

Another key limitation arises from the reliance on signature-based detection within both the Deep Packet Inspection (DPI) and malware detection modules. While this approach ensures rapid identification of known threats, it lacks robustness against zero-day attacks and polymorphic malware, which can dynamically evade static signature databases. This inherent vulnerability was observed in test simulations where modified payloads bypassed detection due to slight alterations in structure. Although sandboxing partially addresses this issue by monitoring runtime behavior, a more effective future direction would be the integration of behavior-based detection and anomaly classification models—such as neural networks or statistical profiling—that can generalize beyond known attack patterns. These adaptive techniques would complement the signature engine, significantly improving resilience against evolving threats.

VI. EVALUATION AND PERFORMANCE METRICS

A. DETECTION ACCURACY

The UTM system's detection rate is measured in terms of its ability to detect phishing attacks, malware (viruses, worms, trojans), and network intrusions. The accuracy is measured as the number of threats identified correctly divided by the number of threats encountered while in test. High detection accuracy is important in terms of being able to offer strong network security.

B. FIREWALL EFFECTIVENESS

Firewall performance is quantified in terms of throughput and latency measurements. Throughput is the maximum level at which data can be passed through the firewall without delay or loss. Latency is the time it takes the firewall to scrutinize and forward packets. Good firewall performance ensures that security features do not significantly impact network speed or user experience.

C. SYSTEM RESOURCE UTILIZATION

Resource utilization of the UTM system by CPU, memory, and disk I/O under varying loads is gauged. Proper resource utilization is necessary in order to achieve stability, especially when the various UTM services (such as IDS/IPS, antivirus scans, and content filtering) are run simultaneously.

D. FALSE POSITIVES AND FALSE NEGATIVES

The consistency of the threat detection mechanisms is also ensured by the following parameters: False Positives (FP): Instances where legitimate traffic is detected as malicious. False Negatives (FN): Instances where actual threats are not identified. The best UTM system will try to reduce both the FP and FN rates to increase accuracy and trust among users.

TABLE 2. Classification report of phishing detection model.

Class	Precision	Recall	F1-Score	Support
0 (Legitimate)	0.97	0.97	0.97	1697
1 (Phishing)	0.97	0.97	0.97	1732
Accuracy	—	—	0.97	3429
Macro Avg	0.97	0.97	0.97	3429
Weighted Avg	0.97	0.97	0.97	3429

TABLE 3. Firewall performance under load.

Load (Mbps)	Throughput	Latency
100	98.7%	1.5 ms
500	96%	2.9 ms
1000	94.5%	4.8 ms

VII. RESULTS AND DISCUSSION

The test results of the Unified Threat Management (UTM) system indicate excellent performance on some key parameters. The discussion below explains these results and identifies the efficiency and limitation.

A. THREAT DETECTION EFFECTIVENESS

The UTM system performed well in cyber threat detection with 97.0% phishing detection accuracy and 0.996 AUC-ROC. The above was attained employing an ensemble model that combined Random Forest, XGBoost, LightGBM, Catboost, trained and tested on the Phishing Websites Dataset. The ensemble successfully employed primary lexical and behavioural features to identify malicious URLs from benign URLs. While these results are promising, observe that the detection performance of phishing was evaluated on a static labelled test set. Real-world phishing URLs are likely to change with fresh obfuscation and dynamic behaviours not covered by such test sets. The responsiveness and generalizability of the system to fresh threats in real-world settings are therefore areas for future work. Table 2 shows the classification report of the model.

B. FIREWALL PERFORMANCE

As summarized in Table 3, Firewall performance was well over 94% effective even at 1000 Mbps load, with the latency increasing only by 1.5 ms to 4.8 ms when traffic increased. From the test results, it is clear that the firewall module is quite capable of withstanding high-traffic scenarios without acting as a bottleneck. This proves useful when implemented in a corporate setup where real-time threat protection and round the-clock data exchange is a priority.

C. SYSTEM RESOURCE USAGE

As summarized in Table 4. During peak operational load—where intrusion detection, antivirus scanning, and content filtering were active—the CPU and memory usage peaked at 75% and 70%, respectively. These values indicate efficient use of system resources and suggest that the system can sustain long-term operations without degradation or crashes, even under stress.

TABLE 4. Peak system resource usage.

Component	Peak Usage
CPU	75%
Memory	70%

TABLE 5. False positive and false negative rates.

Metric	Value (%)
False Positive Rate	2.3
False Negative Rate	3.1

TABLE 6. Proposed UTM system vs Fortinet UTM.

Metric	Proposed UTM System	FortinetUTM(%)
Detection Accuracy	97.0%	94.5%
Firewall throughput	94.5%	93%
Latency	4.8ms	5-7ms
Adaptive Learning	ML(Ensemble),NLP,DPI	Rule Based

D. RELIABILITY OF DETECTION MECHANISM

As summarized in Table 5. The system recorded a false positive rate of 2.3% and a false negative rate of 3.1%, both of which are acceptable industry levels. The low false positive rate minimizes unwanted alarms and increases operational efficiency, and the low false negative rate guarantees that most threats were effectively identified. Bringing these rates even lower would increase the trust and effectiveness of the UTM solution.

E. COMPARISON WITH EXISTING SYSTEM

To help readers assess the practical relevance of our proposed UTM system, Table 6 provides a direct comparison with Fortinet, one of the leading commercial UTM solutions. While Fortinet offers mature security features and proprietary AI models, our system demonstrates competitive accuracy, lower latency, and a more modular, transparent architecture, particularly suitable for academic, SME, and open research deployments [18], [19], [20], [21].

F. OBSERVATION AND LIMITATION

The UTM system performs well in identifying typical threat patterns and handling high volumes of traffic. The system can be made ineffective if it is executed on low-end machines or in low-resource environments. Even though they are rare, false negatives underscore the call for more anomaly detection capacity, potentially through incorporating machine learning. The test also did not take into account zero-day attacks and adaptive evasion techniques, which are a major consideration in current threat landscapes.

VIII. CONCLUSION

The work provided a comprehensive Unified Threat Management (UTM) system integrating a number of security modules like intrusion detection, firewall, antivirus, and content filtering. Strong detection rates against phishing attacks along with efficient firewall throughput and negligible latency were

recorded while the network was subjected to heavy loads under ensemble learning techniques. Resource usage stayed within desired limits, while false positive to false negative ratios were low. The findings corroborate the suitability of the developed UTM system for deployment within today's enterprise networks. Although it has its strengths, future studies could involve the incorporation of zero-day threat detection capabilities, real-time adaptive learning, and cloud-native scalability to further enhance the system to be more responsive and resilient.

Integrating adaptive zero-day attack detection mechanisms, potentially leveraging generative adversarial networks (GANs) or continual learning frameworks to identify novel threat signatures in real time. Additionally, incorporating dynamic learning modules—such as online machine learning or reinforcement learning—could allow the UTM system to evolve with emerging attack patterns without requiring complete retraining. These improvements would significantly increase the responsiveness and long-term resilience of the system in high-risk, rapidly changing threat environments.

DECLARATIONS

CONFLICT OF INTEREST: The authors declare that they have no conflict of interest.

Ethics approval and consent to participate: Not applicable.

Consent for publication: Not applicable.

Data availability: The phishing detection model was trained and evaluated using the publicly available Phishing Websites Dataset (Version 3) available at <https://data.mendeley.com/datasets/c2gw7fy2j4/3> on Mendeley Data.

Materials availability: Not applicable.

Code availability: The code developed for the Unified Threat Management (UTM) system is available from the corresponding author upon reasonable request.

Author contribution: All authors contributed equally to the design, implementation, evaluation, and writing of the manuscript.

REFERENCES

- [1] A. Hussain, A. Mohamed, and S. Razali, "A review on cybersecurity: Challenges & emerging threats," in *Proc. 3rd Int. Conf. Netw., Inf. Syst. Secur.*, Mar. 2020, pp. 1–7.
- [2] J. Jang-Jaccard and S. Nepal, "A survey of emerging threats in cybersecurity," *J. Comput. Syst. Sci.*, vol. 80, no. 5, pp. 973–993, 2014.
- [3] G. Tsochev, R. Trifonov, O. Nakov, S. Manolov, and G. Pavlova, "Cyber security: Threats and challenges," in *Proc. Int. Conf. Automatics Informat. (ICAI)*, Oct. 2020, pp. 1–6.
- [4] J. Gharat, A. Vidhate, and A. Barve, "Performance analysis of unified threat management (UTM)," in *Inventive Communication and Computational Technologies*, G. Ranganathan, J. Chen, and A. Rocha, Eds., Singapore: Springer, 2020, pp. 121–133, doi: [10.1007/978-981-15-0146-3_13](https://doi.org/10.1007/978-981-15-0146-3_13).
- [5] M. Schmitt, "Securing the digital world: Protecting smart infrastructures and digital industries with artificial intelligence (AI)-enabled malware and intrusion detection," 2023, *arXiv:2401.01342*.
- [6] K. Cabaj, Z. Kotulski, B. Książkowski, and W. Mazurczyk, "Cybersecurity: Trends, issues, and challenges," *EURASIP J. Inf. Secur.*, vol. 2018, no. 1, pp. 1–3, Dec. 2018, doi: [10.1186/s13635-018-0080-0](https://doi.org/10.1186/s13635-018-0080-0).
- [7] R. S. Nimbalkar and B. B. Meshram, "Survey on integrated threat management (ITM)," *Int. J. Eng. Res. Technol.*, vol. 2, no. 6, pp. 290–297, 2013.

- [8] J. Zhang and M. Xie, "A review of unified threat management (UTM) systems," *J. Cybersec. Priv.*, vol. 1, no. 3, pp. 123–145, 2019.
- [9] T. L. Lau, P. P. Man, and L. W. Chai, "Evaluation of unified threat management (UTM) systems against advanced persistent threats," *Comput. Secur.*, vol. 83, pp. 177–190, Jan. 2019, doi: [10.1016/j.cose.2019.02.010](https://doi.org/10.1016/j.cose.2019.02.010).
- [10] Y. Zhang, H. Wang, and Z. Li, "Comparative analysis of UTM systems for small and medium-sized enterprises," *J. Inf. Secur.*, vol. 11, no. 5, pp. 409–421, 2020.
- [11] S. Bilgin and O. Bektas, "Performance optimization in UTM systems," *Int. J. Comput. Appl.*, vol. 182, no. 5, pp. 18–26, 2020.
- [12] L. Yuan, H. Chen, J. Mai, C.-N. Chuah, Z. Su, and P. Mohapatra, "FIREMAN: A toolkit for firewall modeling and analysis," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, May 2006, pp. 15 pp.–213. [Online]. Available: <https://scholar.google.com/citations?viewop=listclassicarticles&hl=en&by=2006&vq=engcompsecuritycryptology>
- [13] A. Sharma, D. Verma, N. Sharma, and N. Jindal, "A review of cyber security and its approaches with recent progress and challenges," *Int. J. Eng. Technol. Manage. Sci.*, vol. 7, no. 6, pp. 186–193, 2023, doi: [10.46647/ijetms.2023.v07i06.029](https://doi.org/10.46647/ijetms.2023.v07i06.029).
- [14] F. Cremer, B. Sheehan, M. Fortmann, A. N. Kia, M. Mullins, F. Murphy, and S. Materne, "Cyber risk and cybersecurity: A systematic review of data availability," *Geneva Papers Risk Insurance-Issues Pract.*, vol. 47, no. 3, pp. 698–736, Jul. 2022.
- [15] L. Ofusori, T. Bokaba, and S. Mhlomo, "Artificial intelligence in cybersecurity: A comprehensive review and future direction," *Appl. Artif. Intell.*, vol. 38, no. 1, 2024, doi: [10.1080/08839514.2024.2439609](https://doi.org/10.1080/08839514.2024.2439609).
- [16] N. Jovanovic, C. Kruegel, and E. Kirda, "Pixy: A static analysis tool for detecting web application vulnerabilities," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, May 2006, p. 6.
- [17] F. Hao, R. Anderson, and J. Daugman, "Combining Crypto with Biometrics Effectively," *IEEE Trans. Comput.*, vol. 55, no. 9, pp. 1081–1088, Sep. 2006. [Online]. Available: <https://scholar.google.com/citations?view=listclassicarticles&hl=en&by=2006&vq=engcompsecuritycryptology>
- [18] (2022). *AV-Comparatives UTM Test Reports*. [Online]. Available: <https://www.av-comparatives.org/tests/anti-phishing-certification-fortinet-2025/>
- [19] A. Pasdar, N. Koroniotis, M. Keshk, N. Moustafa, and Z. Tari, "Cybersecurity solutions and techniques for Internet of Things integration in combat systems," *IEEE Trans. Sustain. Comput.*, vol. 10, no. 2, pp. 345–365, Mar. 2024.
- [20] A. Siddiqui, B. P. Rimal, M. Reisslein, D. Gc, and Y. Wang, "SUTMS: Designing a unified threat management system for home networks," *IEEE Access*, vol. 12, pp. 80930–80949, 2024.
- [21] A. Siddiqui, B. P. Rimal, M. Reisslein, and Y. Wang, "Survey on unified threat management (UTM) systems for home networks," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2459–2509, 4th Quart., 2024.



SHREYASH HATKAR was born in Nagpur, India, in 2003. He is currently pursuing the bachelor's degree with Vellore Institute of Technology, Chennai, India. He has completed internships focused on data science and cybersecurity, where he gained hands-on experience with machine learning models, network analysis, and threat detection tools. He has contributed to research in unified threat management and anomaly detection using AI-based techniques. He has also worked on academic projects involving secure data storage, real-time analytics, and NoSQL database systems. His technical interests include data science, cybersecurity, artificial intelligence, and cloud computing. He is passionate about developing secure, intelligent systems that solve real-world problems and aims to pursue higher studies or a research-driven role in industry.



KREET ROUT was born in Ranchi, India, in 2003. He is currently pursuing the bachelor's degree with Vellore Institute of Technology, Chennai, India. He has completed internships focused on data science and cybersecurity, where he gained hands-on experience with machine learning models, network analysis, and threat detection tools. He has contributed to research in unified threat management and anomaly detection using AI-based techniques. He has also worked on academic projects involving secure data storage, real-time analytics, and NoSQL database systems. His technical interests include data science, cybersecurity, artificial intelligence, and cloud computing. He is passionate about developing secure, intelligent systems that solve real-world problems and aims to pursue higher studies or a research-driven role in industry.



AAYUSH LAD was born in Valsad, India, in 2003. He is currently pursuing the bachelor's degree with Vellore Institute of Technology, Chennai, India. He has completed internships focused on data science and cybersecurity, where he gained hands-on experience with machine learning models, network analysis, and threat detection tools. He has contributed to research in unified threat management and anomaly detection using AI-based techniques. He has also worked on academic projects involving secure data storage, real-time analytics, and NoSQL database systems. His technical interests include data science, cybersecurity, artificial intelligence, and cloud computing. He is passionate about developing secure, intelligent systems that solve real-world problems and aims to pursue higher studies or a research-driven role in industry.



O. V. GNANA SWATHIKA (Senior Member, IEEE) received the B.E. degree in electrical and electronics engineering from the University of Madras, Chennai, India, in 2000, the M.S. degree in electrical engineering from Wayne State University, Detroit, MI, USA, in 2004, the Ph.D. degree in electrical engineering from Vellore Institute of Technology (VIT), Chennai, in 2017, and the Post-doctoral degree from the University of Moratuwa, Sri Lanka, in 2019. She is currently a Professor with the Centre for Smart Grid Technologies, VIT. Her current research interests include microgrid protection, power system optimization, embedded systems, and photovoltaic systems.

...